

Documento de arquitectura

Billetera: contenedor Android en Kotlin con cuatro bundles de React Native

JHON ALEXANDER SUAREZ OSSA

Prueba técnica · Desarrollador React Native

Septiembre de 2026

Contenido

1. Introducción	5
1.1 Qué pidió la prueba.....	5
1.2 Qué se entregó.....	6
1.3 Vocabulario mínimo	6
1.4 Versiones con las que está construido y probado	6
2. Visión general de la solución.....	8
2.1 Cómo se reparte el trabajo	8
2.2 Componentes, responsabilidad y ubicación en el código	8
2.3 Un host de React Native por pantalla	12
2.4 Cómo llegan los bundles al APK	12
3. Decisiones de arquitectura.....	14
3.1 React Native 0.81.6 con arquitectura clásica	14
3.2 Un host de React Native por pantalla, en vez de uno compartido	14
3.3 Almacenamiento cifrado propio en vez de Jetpack Security	14
3.4 minSdk 26.....	15
3.5 Backend simulado en proceso.....	15
3.6 Un solo sobre cifrado para todo el puente	16
3.7 Otras decisiones menores	16
4. Comunicación entre Android y React Native	18
4.1 Props iniciales: Android entrega, una sola vez.....	18
4.2 El módulo nativo: JavaScript pregunta, Android responde.....	18
Manejo de errores del módulo	19
4.3 Los eventos: Android empuja, o el bundle avisa.....	19
Dos decisiones sobre los eventos que conviene sustentar	20
4.4 Recorrido 1: del arranque al saldo en Home	21
4.5 Recorrido 2: una transferencia.....	23
4.6 Recorrido 3: la sesión que vence.....	24
5. Manejo de sesión	26
5.1 Qué se guarda	26
5.2 Dónde y cómo se guarda.....	26

5.3 Cuándo se valida	26
5.4 Cuándo expira	27
5.5 Qué pasa al cerrar sesión	27
6. Cifrado	28
6.1 El cifrado del almacenamiento	28
6.2 El sobre cifrado del canal	28
6.3 Interoperabilidad comprobada con un vector compartido	29
6.4 Qué se cifra y qué no.....	30
6.5 Modelo de amenazas del canal.....	30
Qué sí protege	31
Qué no protege	31
Cuál sería el siguiente paso	31
7. Seguridad implementada, punto por punto	32
7.1 Notas sobre tres de esos puntos.....	33
7.2 Dos medidas adicionales que no estaban en la lista.....	34
8. El backend simulado y sus reglas de negocio.....	35
8.1 Cómo está construido	35
8.2 Las reglas	35
Login	35
Registro	36
Transferencia.....	36
Movimientos	37
8.3 Datos semilla	37
8.4 Una nota sobre enumeración de cuentas.....	38
9. Estrategia de pruebas.....	39
9.1 Cobertura de las once pruebas mínimas.....	39
9.2 Lo que no se pudo probar	40
10. Limitaciones conocidas y próximos pasos.....	41
10.1 Limitaciones	41
10.2 Próximos pasos, en orden	41
10.3 Dos desviaciones respecto de lo planeado	42
Anexo. Índice de figuras y archivos de referencia	43

1. Introducción

1.1 Qué pidió la prueba

La prueba pedía una aplicación Android nativa escrita en **Kotlin** que cargara cuatro pantallas construidas en **React Native**, con comunicación cifrada entre las dos mitades, manejo seguro de sesión y las prácticas de seguridad que se le exigen a una aplicación de una entidad financiera.

El reparto de tecnologías venía fijado en el enunciado. La tabla siguiente lo repite y agrega, en la tercera columna, la pieza concreta del proyecto que resolvió cada casilla.

Componente	Tecnología exigida	Cómo quedó resuelto
Contenedor principal	Android Kotlin	BilleteraApp como Application, más una Activity por pantalla sobre la clase base BundleActivity
Splash	Android Kotlin	SplashActivity, con layout XML nativo y sin una sola línea de JavaScript
Login	React Native Bundle	login.android.bundle, componente LoginBundle
Home	React Native Bundle	home.android.bundle, componente HomeBundle
Transferencia	React Native Bundle	transferencia.android.bundle, componente TransferenciaBundle
Movimientos	React Native Bundle	movimientos.android.bundle, componente MovimientosBundle
Sesión y seguridad	Android Kotlin	SesionManager, SecureStore, LlaveKeystore, AesGcm, PasswordHasher y DeviceIntegrity
Comunicación	Bridge, props, EventEmitter	Props iniciales por getLaunchOptions(), módulo nativo BilleteraBridge y eventos por DeviceEventEmitter
Almacenamiento seguro	EncryptedSharedPreferences o Android Keystore	SecureStore, un almacén propio que cifra cada valor con una llave AES-256 que vive en el Android Keystore
Backend mock / API	Libre	BancoMockApi sobre MockDb: backend simulado en proceso, con estado persistido cifrado

1.2 Qué se entregó

El repositorio cubre la lista completa de entregables del enunciado:

- Código fuente de Android en Kotlin: 32 archivos bajo `android/app/src/main/java/com/asuarez/billetera/`.
- Código fuente de React Native en TypeScript, bajo `src/` y `entries/`.
- Los cuatro bundles ya compilados a bytecode, dentro del APK en `android/app/src/main/assets/bundles/`.
- README con instrucciones de compilación, usuarios de prueba y decisiones principales.
- APK de release, firmado con llave propia y ofuscado con R8.
- Este documento de arquitectura, los cinco diagramas de `docs/diagramas/` y la evidencia de pruebas en `docs/evidencia.md`.

1.3 Vocabulario mínimo

El documento está escrito para que se pueda leer sin conocer React Native. Estos cinco términos aparecen en todas las secciones y conviene fijarlos de entrada.

- **Bundle:** un archivo con todo el JavaScript de una pantalla ya empaquetado. Cumple el mismo papel que un `.jar` o un `.dex`, pero para JavaScript.
- **Hermes:** el motor de JavaScript que trae React Native. Aquí los bundles no son texto plano: están compilados a bytecode de Hermes, igual que el código Java se compila a bytecode de la máquina virtual.
- **Puente (bridge):** el canal de mensajes entre Kotlin y JavaScript. Las dos mitades corren en el mismo proceso, pero no comparten memoria: se pasan cadenas de texto.
- **Módulo nativo:** una clase de Kotlin cuyos métodos JavaScript ve como funciones que devuelven promesas. Es la única puerta de JavaScript hacia Android.
- **Props iniciales:** los datos que Android le entrega al componente de React en el momento de montarlo. Llegan una sola vez, al arrancar la pantalla, y ya no se pueden pedir de nuevo.

Conviene aclarar algo desde el principio, porque cambia la lectura de todo lo demás: aquí no hay servidor de desarrollo, no hay descarga de bundles por red y no hay Metro. Cada pantalla carga un archivo `.android.bundle` que ya viene dentro del APK firmado.

1.4 Versiones con las que está construido y probado

Pieza	Versión
React Native	0.81.6, con arquitectura clásica (<code>newArchEnabled=false</code>)
React	19.1.4
Motor de JavaScript	Hermes (<code>hermesEnabled=true</code>)
Kotlin	2.1.20

Android Gradle Plugin	8.11.0, el que fija el plugin de Gradle de React Native
Gradle	8.14.3, con el wrapper del repositorio
JDK	17 (Temurin 17.0.20)
SDK de Android	minSdk 26, compileSdk y targetSdk 36, build-tools 36.0.0
Node	24.18.0 con npm 11.19.0
Equipo de prueba	Emulador Pixel 6, API 36, imagen googleApis x86_64

2. Visión general de la solución

2.1 Cómo se reparte el trabajo

Android manda. Decide la pantalla de entrada, controla la navegación, verifica las claves, crea y borra la sesión, custodia las llaves criptográficas y ejecuta las reglas de negocio. Los bundles de React Native pintan la interfaz, validan lo obvio en pantalla para no cruzar el puente por gusto y le piden a Android todo lo demás.

Ningún bundle tiene almacenamiento propio, ni acceso al banco simulado, ni forma de navegar por su cuenta. Un bundle que se ejecutara suelto no podría hacer nada: todo lo que necesita entra y sale por el módulo nativo.

Billetera. Diagrama de componentes

App Android en Kotlin que hospeda cuatro pantallas hechas en React Native. Android manda en la navegación, la sesión y las llaves.

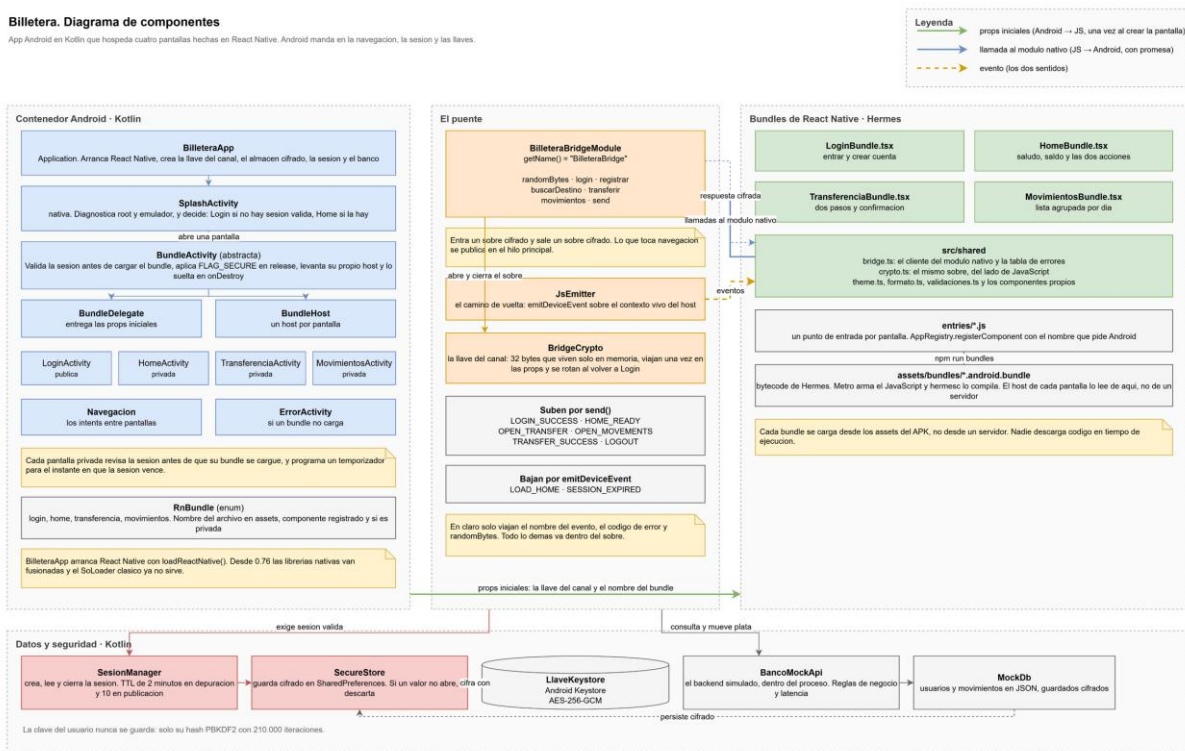


Figura 1. Diagrama de componentes. A la izquierda el contenedor de Kotlin, en el centro los tres mecanismos del puente y a la derecha los cuatro bundles de React Native. Abajo, dónde queda guardado el estado.

2.2 Componentes, responsabilidad y ubicación en el código

Las rutas de la tercera columna son relativas a android/app/src/main/java/com/asuarez/billetera/ para el lado Kotlin, y a la raíz del repositorio para el lado JavaScript.

Componente	De qué responde	Dónde está
BilleteraApp	Carga las librerías nativas de React Native,	BilleteraApp.kt

	genera la llave del canal y arma las dos instancias compartidas de toda la aplicación: SesionManager y BancoMockApi.	
SplashActivity	Pantalla nativa de entrada. Corre el diagnóstico de root y emulador fuera del hilo de interfaz, aplica la política de bloqueo o de aviso y decide si sigue a Login o a Home.	splash/SplashActivity.kt
BundleActivity	Clase base de las cuatro pantallas de React Native. Levanta el host, exige sesión en las pantallas privadas, programa el vencimiento, aplica FLAG_SECURE en release, recibe los eventos que suben del bundle y emite los que bajan.	bundles/BundleActivity.kt
BundleDelegate	Le dice a React Native qué host usar, qué componente montar y qué props iniciales entregar.	bundles/BundleDelegate.kt
BundleHost	Un DefaultReactNativeHost por Activity. Apunta al .android.bundle correspondiente, registra el paquete del puente y redirige los fallos de JavaScript a la pantalla de error.	bundles/BundleHost.kt
RnBundle	Enumeración con los cuatro bundles:	bundles/RnBundle.kt

	nombre de archivo, nombre del componente registrado y si la pantalla es privada.	
LoginActivity, HomeActivity, TransferenciaActivity, MovimientosActivity	Cada una atiende su bundle y atiende sus propios eventos. Home es la única con estado real: el saludo de enlace y el reenvío del saldo.	bundles/*.kt
BilleteraBridgeModule	El módulo nativo. Única puerta de JavaScript hacia Android: login, registro, búsqueda de destino, transferencia, movimientos, randomBytes y el send de eventos.	bridge/BilleteraBridgeModule.kt
JsEmitter	El camino contrario: emite eventos de Android hacia el bundle activo, cifrando el payload.	bridge/JsEmitter.kt
BridgeCrypto	Llave del canal (32 bytes en memoria), cifrado y descifrado de los sobres, y rotación al volver a Login.	security/BridgeCrypto.kt
LlaveKeystore, AesGcm, SecureStore	Llave AES-256 en el Android Keystore, AES-GCM sobre javax.crypto y el almacén clave/valor cifrado sobre SharedPreferences.	security/*.kt
PasswordHasher	PBKDF2WithHmacSHA256 para las claves de usuario.	security/PasswordHasher.kt
DeviceIntegrity	Heurísticas locales de root y de emulador, con la lista de señales que	security/DeviceIntegrity.kt

	se le muestra al usuario.	
SesionManager y Sesion	Ciclo de vida de la sesión: crear, leer, validar expiración y cerrar.	session/*.kt
BancoMockApi, MockDb, DatosSemilla	El backend simulado en proceso, con sus reglas de negocio y su estado persistido cifrado.	data/*.kt
Navegacion	Los cinco intents de la aplicación en un solo sitio, cada uno con sus flags.	nav/Navegacion.kt
ErrorActivity	Pantalla nativa de rescate cuando un bundle no carga o falla en ejecución.	error/ErrorActivity.kt
bridge.ts	Cliente tipado del módulo nativo del lado de JavaScript: métodos, suscripción a eventos y traducción de códigos de error a texto.	src/shared/bridge.ts
crypto.ts	La otra mitad del cifrado del canal, con @noble/ciphers y @scure/base.	src/shared/crypto.ts
Los cuatro bundles	Las pantallas: Login con registro, Home, Transferencia con su máquina de pasos y Movimientos con su agrupación por día.	src/bundles/{login,home,transferencia,movimientos}/
build-bundles.js	Empaqueta con Metro y compila con hermes los cuatro .android.bundle.	scripts/build-bundles.js

2.3 Un host de React Native por pantalla

Un detalle define la forma del contenedor: **cada Activity levanta su propio host de React Native y lo destruye al salir**. No hay un motor de JavaScript compartido entre las cuatro pantallas.

El precio son unos cientos de milisegundos al abrir cada pantalla y más memoria mientras hay dos vivas a la vez, que son Home y su pantalla hija. A cambio, cada bundle queda aislado de verdad: el estado de Transferencia no sobrevive a la pantalla, y si un bundle falla se lleva su host y nada más. Como el enunciado pide cuatro bundles separados, esa separación es justamente el punto.

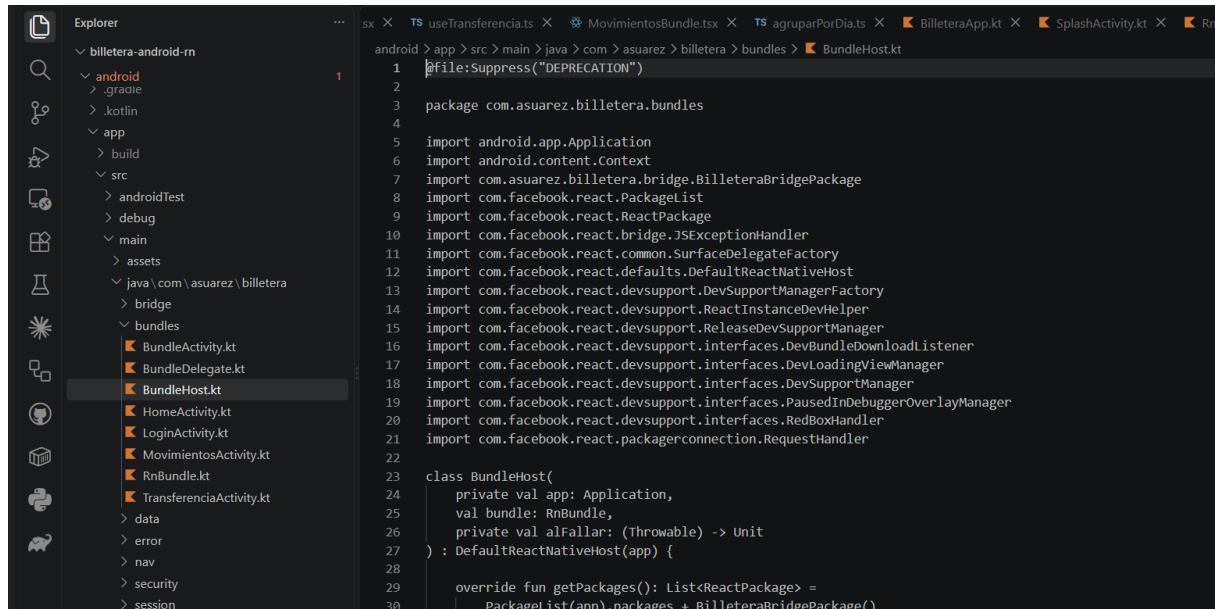


Figura 2. Figura 2. `BundleHost` extiende `DefaultReactNativeHost`. El método `getBundleAssetName()` es lo que ata cada Activity a su propio archivo dentro de assets.

2.4 Cómo llegan los bundles al APK

Cada pantalla se registra con AppRegistry en un archivo de entries/. El script `npm run bundles` empaqueta ese punto de entrada con Metro y pasa el resultado por hermes, que lo compila a bytecode.

```

npm run bundles

# por cada uno de los cuatro nombres:
# 1. metro:  --dev false --minify false
# 2. hermes: -O -emit-binary
# 3. queda en assets/bundles/<nombre>.android.bundle
  
```

El `--minify false` de Metro no deja texto suelto en el APK, porque la minificación la hace Hermes al compilar a bytecode. Los cuatro archivos resultantes pesan alrededor de 1,6 MB cada uno y están commiteados a propósito: el enunciado los pide como entregable y sin ellos el APK no arranca.

3. Decisiones de arquitectura

Cada decisión se presenta igual: el problema que había que resolver, la opción elegida, la alternativa que se descartó y el motivo.

3.1 React Native 0.81.6 con arquitectura clásica

Problema. React Native tiene dos arquitecturas vivas. La clásica, basada en el puente de mensajes, y la nueva (bridgeless, con TurboModules y el renderizador Fabric). Había que escoger una antes de escribir la primera línea.

Decisión. Arquitectura clásica, con `newArchEnabled=false` en `gradle.properties`.

Alternativa descartada. La nueva arquitectura, que es la dirección oficial del proyecto y la que traerán las versiones futuras.

Por qué. El enunciado nombra por su nombre el React Native Bridge, los Native Modules y el EventEmitter, que son exactamente las piezas de la arquitectura clásica. Además, la nueva arquitectura no admite bien el patrón de un `ReactNativeHost` propio por Activity, que es lo que hace falta para tener bundles realmente separados. Con `newArchEnabled=false`, el `ReactActivityDelegate` clásico deja sobrescribir `getReactNativeHost()`, `getLaunchOptions()` y `getMainComponentName()`, que es todo lo necesario. El costo es depender de APIs marcadas como obsoletas, y por eso los tres archivos de hosting llevan la anotación de supresión en la cabecera, en lugar de repartir anotaciones sueltas por el código.

3.2 Un host de React Native por pantalla, en vez de uno compartido

Problema. Levantar el motor de JavaScript cuesta tiempo y memoria. Una aplicación de producción normalmente levanta uno solo y lo reutiliza para todas las pantallas.

Decisión. Un `BundleHost` por Activity, creado de forma perezosa y liberado en `onDestroy`.

Alternativa descartada. Un host único con los cuatro componentes registrados en un mismo bundle.

Por qué. Con un host único habría que meter las cuatro pantallas en un solo archivo, o intercambiar el bundle en caliente con trucos poco defendibles. En los dos casos dejarían de ser cuatro bundles, que es lo que el enunciado pide. Con un host por Activity, Movimientos carga solo el bytecode de Movimientos, y como mucho hay dos hosts vivos al tiempo.

3.3 Almacenamiento cifrado propio en vez de Jetpack Security

Problema. El enunciado pide `EncryptedSharedPreferences` o un equivalente, apoyado en el Android Keystore.

Decisión. `SecureStore`, un almacén clave/valor propio que cifra cada valor con una llave AES-256 del Keystore antes de escribirlo en un `SharedPreferences` privado.

Alternativa descartada. EncryptedSharedPreferences, de la librería androidx.security:security-crypto.

Por qué. Esa clase está marcada como obsoleta desde security-crypto 1.1 y no tiene reemplazo directo. Una dependencia sin mantenimiento dentro de una aplicación financiera es difícil de defender. El equivalente propio, además, se declara como interfaz (KeyValueStore), así que las pruebas de la JVM lo sustituyen por una implementación en memoria y no necesitan emulador.

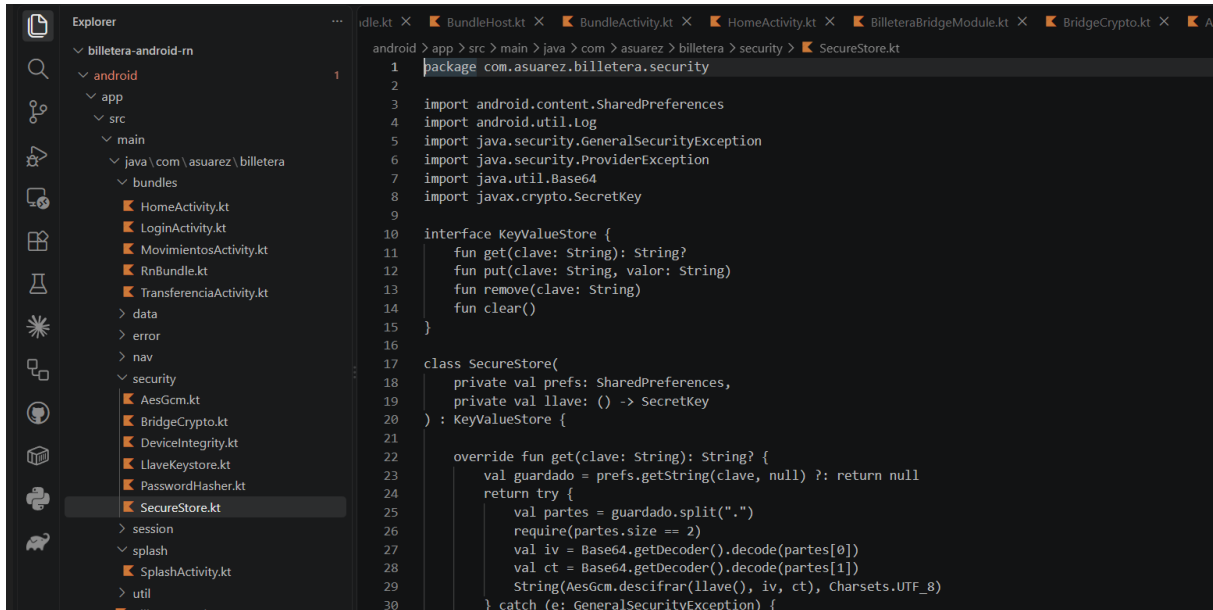


Figura 3. Figura 3. `SecureStore` y la interfaz `KeyValueStore`. Si un valor no abre, se registra el tipo de excepción, se borra la entrada y se devuelve `null`; el contenido no se escribe nunca en el log.

3.4 minSdk 26

Problema. Elegir el Android mínimo soportado. Cada versión que se baja amplía el mercado y añade trabajo de compatibilidad.

Decisión. minSdk 26, es decir Android 8.0.

Alternativa descartada. minSdk 21, que es el mínimo histórico habitual en React Native.

Por qué. Tres piezas de la plataforma que esta aplicación usa a diario aparecen en API 26: PBKDF2WithHmacSHA256, java.time y java.util.Base64. Con 21 haría falta desugaring y una librería de criptografía aparte, a cambio de un tramo de mercado que a estas alturas ya casi no existe. El motivo está escrito como comentario en android/build.gradle, junto a la declaración.

3.5 Backend simulado en proceso

Problema. El enunciado deja libre el backend. Había que decidir si montar un servidor aparte o resolverlo dentro del dispositivo.

Decisión. BancoMockApi, un backend simulado que corre en el mismo proceso, con MockDb guardando usuarios y movimientos en el almacén cifrado.

Alternativa descartada. Un servicio HTTP aparte, propio o alojado, al que la aplicación se conectara por red.

Por qué. Un servidor aparte convierte la evaluación en un montaje: hay que levantarlo, exponerlo y mantenerlo vivo mientras alguien revisa la prueba. El mock en proceso se instala con el APK y funciona sin red. Para que la sustitución posterior sea un reemplazo y no una reescritura, BancoMockApi está escrito con la forma de un cliente HTTP: métodos que devuelven Resultado.Ok o Resultado.Error(codigo, mensaje) en vez de lanzar excepciones para el flujo normal, y latencia simulada de 200 a 400 ms en autenticar y transferir para que las pantallas tengan que manejar estados de carga de verdad.

Consecuencia visible: el APK de release ni siquiera declara el permiso de INTERNET. La aplicación no lo necesita, porque los bundles están en assets y el banco es local. El manifest de depuración de React Native sí lo agrega, junto con SYSTEM_ALERT_WINDOW, que es lo que permite conectar Metro en desarrollo.

3.6 Un solo sobre cifrado para todo el puente

Problema. El enunciado exige cifrar los datos sensibles que cruzan entre Android y los bundles: sesión, datos de usuario, transferencias e identificadores. Había que decidir qué se cifra y con qué formato.

Decisión. Un formato único de sobre AES-256-GCM para todo lo que lleve datos de negocio, en los dos sentidos y en los tres mecanismos: argumentos de los métodos nativos, respuestas de esos métodos y payloads de evento.

Alternativa descartada. Cifrar solo los campos considerados sensibles, dejando el resto del objeto en claro.

Por qué. Decidir campo por campo qué es sensible es una fuente permanente de olvidos: basta agregar un campo nuevo sin acordarse de la regla. Con un solo sobre, la regla es trivial de auditar y de probar. En claro quedan únicamente los nombres de evento, los códigos de error, la respuesta de randomBytes y las props iniciales, que son la excepción obligada porque ahí viaja la llave con la que se abre todo lo demás.

3.7 Otras decisiones menores

Decisión	Alternativa descartada	Motivo
org.json para serializar	Gson, Moshi o kotlinx-serialization	Los payloads son pequeños y planos. org.json viene en el SDK, tiene una implementación real en Maven Central para las pruebas de JVM y evita una dependencia más en una aplicación cuyo

		punto es la seguridad, no el modelado de datos.
PBKDF2 para las claves	Argon2 o bcrypt	PBKDF2WithHmacSHA256 viene en la plataforma desde API 26. Argon2 exige una librería nativa y bcrypt otra dependencia; para un mock que corre en el teléfono no se justifican.
Bundles compilados y commiteados	Generarlos en cada compilación de Gradle	El enunciado los pide como entregable y sin ellos el APK no arranca. Los .jsbundle intermedios sí están ignorados, igual que keystore.properties y el .jks de la firma.
Los movimientos se piden por método, no por evento	Un evento LOAD_MOVEMENTS simétrico a LOAD_HOME	Los eventos son para cosas que Android decide: navegar, refrescar, avisar. Una consulta que la pantalla inicia y espera encaja mejor en una promesa.

4. Comunicación entre Android y React Native

El enunciado pedía props iniciales, módulos nativos y EventEmitter. Los tres están implementados y cada uno hace una cosa distinta. Ninguno duplica el trabajo de los otros.

4.1 Props iniciales: Android entrega, una sola vez

Son los datos que Android le pasa al componente de React en el momento de montarlo. Salen de `getLaunchOptions()` en `BundleDelegate` y llegan como las props del componente raíz. Van una sola vez, al arrancar la pantalla.

```
{
  "claveCanal": "<base64 de 32 bytes>",
  "bundle": "home",
  "esDebug": true,
  "version": "1.0",
  "motivo": "expirada"
}
```

El campo `motivo` solo aparece en Login, y solo si el intent lo traía, con el valor `logout` o `expirada`. Lo importante de este canal es que **la llave del canal viaja por aquí**: es el único momento en que JavaScript la recibe, y no hay forma de pedirla después.

4.2 El módulo nativo: JavaScript pregunta, Android responde

`BilleteraBridgeModule` expone métodos que en JavaScript se ven como funciones que devuelven promesas. Entra un sobre cifrado y sale un sobre cifrado. Las únicas excepciones son `randomBytes` y `send`.

Método	Entra (ya descifrado)	Sale (cifrado)	Sesión
<code>randomBytes(n)</code>	solo el número de bytes	base64 de n bytes de <code>SecureRandom</code> , en claro	no
<code>login(sobre)</code>	{celular, clave}	{sessionId, nombre, celular, expiraEn}	no
<code>registrar(sobre)</code>	{nombre, celular, clave}	igual que login	no
<code>buscarDestino(sobre)</code>	{celular}	{nombre}, con el nombre corto de la cuenta	sí
<code>transferir(sobre)</code>	{celular, monto, descripcion?}	{movimientoId, nuevoSaldo, destinoNombre, fecha}	sí
<code>movimientos()</code>	nada	{movimientos: [...]} con id, fecha, tipo, valor, descripción y estado	sí

send(evento, sobre?)	según el evento	nada	según el evento
----------------------	-----------------	------	-----------------

randomBytes existe por una razón concreta: Hermes no trae `crypto.getRandomValues`, así que el vector de inicialización de cada mensaje se lo pide JavaScript al `SecureRandom` de Android. Esa es la razón por la que la función `cifrar()` del lado JavaScript es asíncrona.

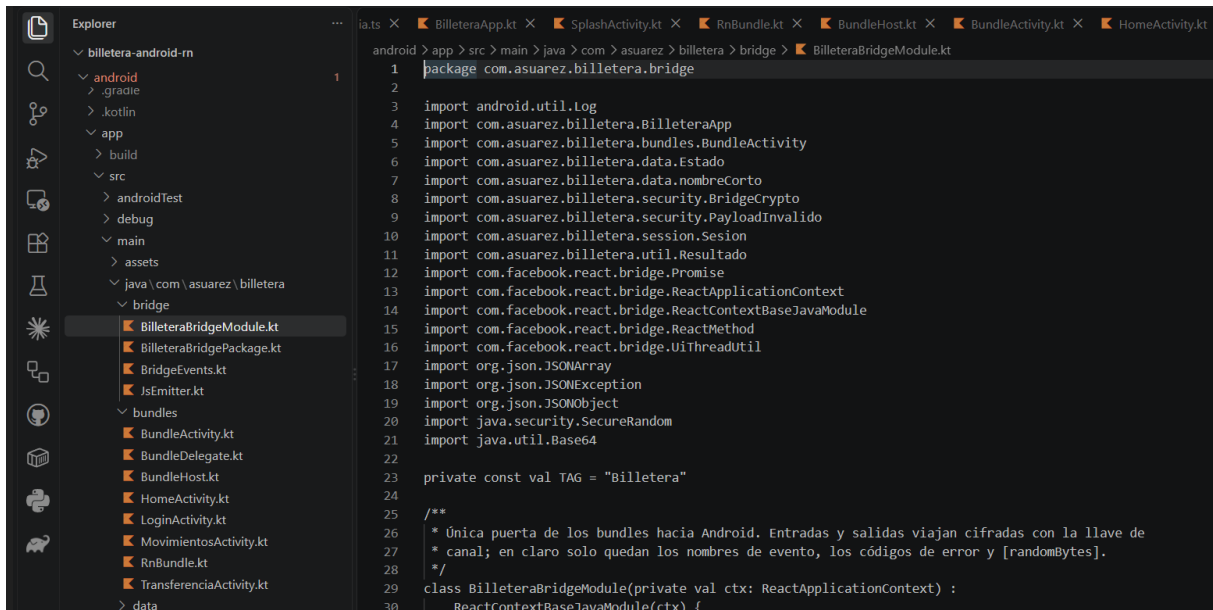


Figura 4. Figura 4. 'BilleteraBridgeModule'. Cada método envuelve su cuerpo en `runCatching`; ninguna excepción sube cruda a JavaScript.

Manejo de errores del módulo

Los errores no suben como excepciones. Cada método rechaza la promesa con un código en mayúsculas y un mensaje: `CREDENCIALES_INVALIDAS`, `SALDO_INSUFICIENTE`, `DESTINO_NO_EXISTE`, `SESION_INVALIDA`, y así.

El lado de JavaScript ignora el mensaje que venga de Android y usa su propia tabla en `bridge.ts`, para que el texto que ve el usuario se decida en un solo lugar. Lo que no esté en esa tabla cae en `ERROR_INTERNO`, con el texto "Ocurrió un error, intenta de nuevo".

4.3 Los eventos: Android empuja, o el bundle avisa

El camino de vuelta. Android emite con `emitDeviceEvent(nombre, sobre)` sobre el contexto de React, y el bundle escucha con `DeviceEventEmitter.addListener`. El payload viaja cifrado igual que en los métodos; si un sobre no descifra, el cliente descarta el evento con una advertencia de una línea y sin volcar el contenido.

La tabla siguiente es la lista completa. Los siete primeros son los que exige el enunciado. `HOME_READY` es un evento agregado, y más abajo se explica qué resuelve.

Evento	Origen → Destino	Contenido	Qué pasa al recibirlo
LOGIN_SUCCESS	Login Bundle → Android	{sessionId}	Android lo compara con la sesión que él mismo guardó. Si coincide, navega a Home. Si no, deja un aviso en el log y no navega.
LOAD_HOME	Android → Home Bundle	{nombre, celular, saldo, expiraEn}	Home pinta el saludo, el celular, la tarjeta de saldo y las dos filas de acción. Se reenvía en cada onResume.
OPEN_TRANSFER	Home Bundle → Android	ninguno	Android revalida la sesión y abre TransferenciaActivity.
OPEN_MOVEMENTS	Home Bundle → Android	ninguno	Android revalida la sesión y abre MovimientosActivity.
TRANSFER_SUCCESS	Transferencia Bundle → Android	{movimientoId}	Se registra en el log. La pantalla la cierra el propio bundle; Home se refresca al reanudar.
LOGOUT	Home Bundle → Android	ninguno	Cierra la sesión, va a Login con motivo "logout" y termina Home. Login rota la llave del canal.
SESSION_EXPIRED	Android → bundle activo	{motivo: "EXPIRADA"}	El bundle muestra el aviso. Android navega a Login dos segundos después.
HOME_READY	Home Bundle → Android	ninguno	Home ya registró sus listeners. Android responde emitiendo LOAD_HOME.

Dos decisiones sobre los eventos que conviene sustentar

`LOGIN\_SUCCESS` se valida contra la sesión. La sesión la crea y la persiste Android dentro de login(), porque Android es quien verifica la clave. LOGIN_SUCCESS es solo la señal de que el bundle terminó su flujo y ya se puede navegar. Por eso LoginActivity compara el sessionId que anuncia el bundle con el de la sesión que él mismo acaba de crear. Un evento suelto nunca abre Home.

`HOME\_READY` funciona como saludo de enlace. Un evento emitido antes de que JavaScript registre su listener se pierde sin dejar rastro. HOME_READY garantiza que LOAD_HOME llegue cuando Home ya está escuchando, y de paso deja abierto el mismo camino para reenviar el saldo al volver de una transferencia. No está en la lista mínima del enunciado; es un evento agregado, y esto es exactamente lo que resuelve.

Hay además dos protecciones pequeñas contra el doble toque, que aparecieron probando en el emulador. HomeActivity tiene una bandera navegando que se limpia en onResume, porque abrir una pantalla hija toma varios cientos de milisegundos y en ese rato Home sigue recibiendo toques; sin eso, dos toques apilan dos Activities o rotan la llave del canal dos veces. Del lado de JavaScript, Home deshabilita las filas mientras espera.

4.4 Recorrido 1: del arranque al saldo en Home

El diagrama recorre el flujo esperado del enunciado de punta a punta: splash nativo, validación de sesión, carga del bundle de Login, verificación de la clave, creación de la sesión cifrada y carga del bundle de Home.

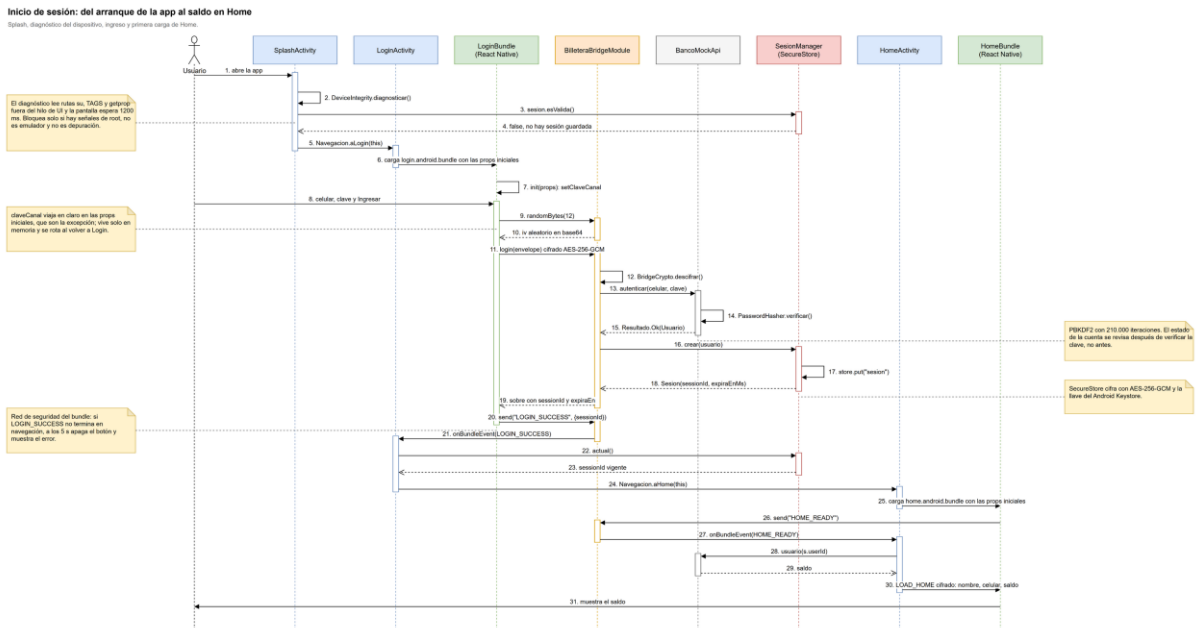
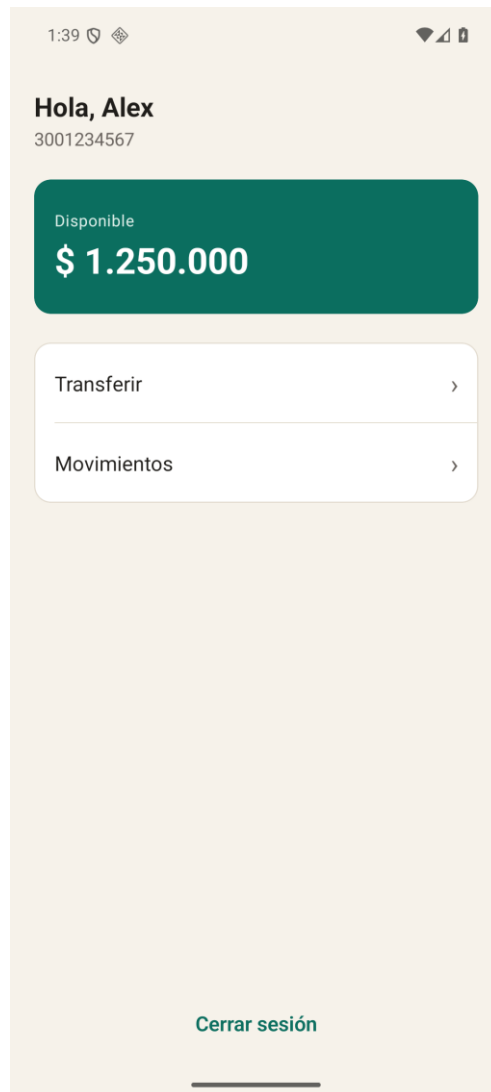


Figura 5. Diagrama de secuencia del inicio de sesión. Arriba, arranque y login. Abajo, el saludo de enlace de Home y la llegada del saldo por 'LOAD_HOME'.

1. SplashActivity se muestra un mínimo de 1,2 segundos y, mientras tanto, corre DeviceIntegrity.diagnosticar() en otro hilo, porque lee archivos y lanza getprop.
2. Si hay señales de root en un dispositivo real y la compilación es de release, la aplicación muestra el diálogo "Dispositivo no seguro" con las señales y se cierra. Si las señales no bloquean (emulador, o root en depuración), muestra un aviso y deja continuar.
3. Con el camino libre, el splash pregunta sesion.esValida(). Si hay sesión vigente va a Home; si no, a Login. Ninguna de las dos rutas pasa por JavaScript.
4. LoginActivity monta el bundle de Login. Antes de eso, si venimos de un logout o de una expiración, rota la llave del canal.
5. El usuario ingresa celular y clave. El bundle valida el formato en pantalla, le pide un vector de inicialización a randomBytes(12), cifra {celular, clave} y llama BilleteraBridge.login(sobre).

6. Android descifra, le pregunta al banco simulado, verifica el hash PBKDF2, revisa que el usuario esté ACTIVO y **crea la sesión ahí mismo**: `SesionManager.crear()` genera el `sessionId`, calcula el vencimiento y guarda el JSON cifrado.
7. Android devuelve al bundle `{sessionId, nombre, celular, expiraEn}`, también cifrado. El bundle envía `LOGIN_SUCCESS` con el `sessionId` que recibió.
8. `LoginActivity` compara ese identificador con el de la sesión que guardó y, si coinciden, navega a Home con `NEW_TASK | CLEAR_TASK`, lo que destruye Login y con ella su host.
9. `HomeActivity` monta el bundle de Home. El bundle se suscribe a `LOAD_HOME` y a `SESSION_EXPIRED`, y recién entonces envía `HOME_READY`.
10. Al recibir `HOME_READY`, Android arma `{nombre, celular, saldo, expiraEn}`, lo cifra y lo emite como `LOAD_HOME`.

El paso 7 tiene una red de seguridad del lado de JavaScript: si el evento no produce navegación en cinco segundos, el botón deja de girar y muestra un error, en vez de quedarse colgado.



Desde Home, la fila "Transferir" envía OPEN_TRANSFER. Android revalida la sesión y abre TransferenciaActivity, que monta su bundle. La pantalla trabaja en dos pasos más una hoja de confirmación.

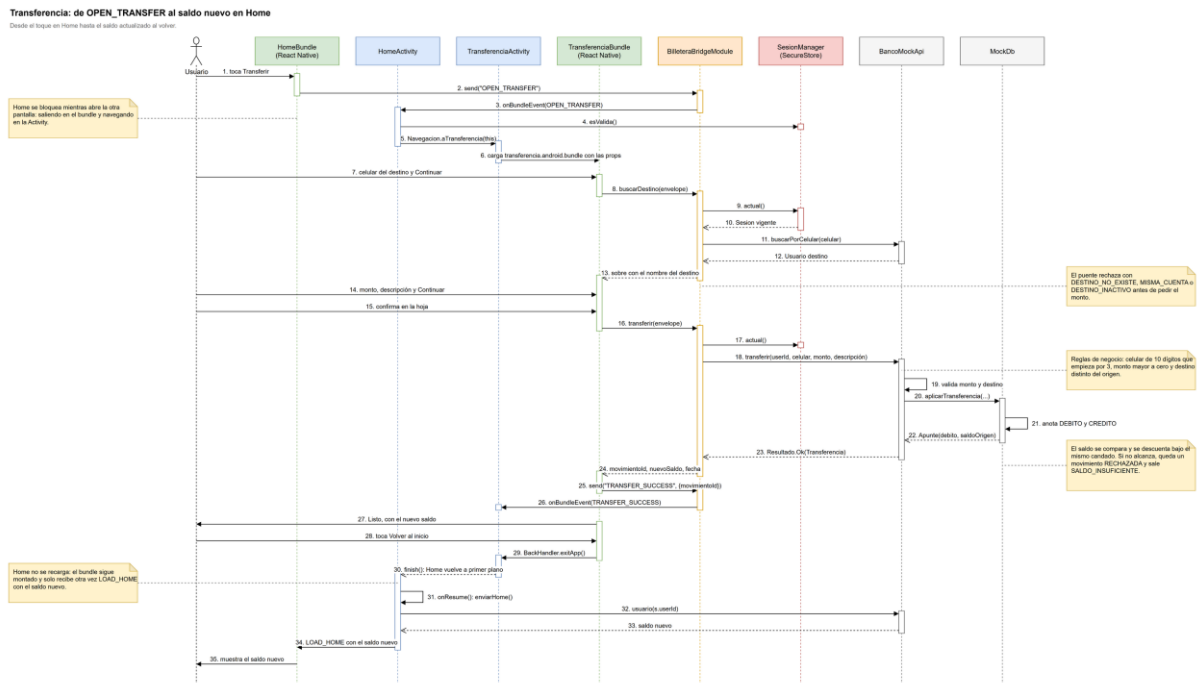


Figura 7. Diagrama de secuencia de una transferencia. Quien valida sesión, destino, monto y saldo es Android; el bundle pregunta y pinta lo que le devuelven.

11. Paso 1 de 2: el usuario escribe el celular destino. El bundle llama buscarDestino, que exige sesión y devuelve el nombre corto de la cuenta, por ejemplo "Laura P.". Si el celular no existe, está inactivo o es el propio, el error viene con su código y la pantalla lo traduce.
12. Paso 2 de 2: monto y descripción opcional. El monto se formatea con separadores de miles mientras se escribe y se valida entero mayor que cero.
13. La hoja de confirmación repite destino y monto. Al confirmar, el bundle cifra {celular, monto, descripcion} y llama transferir.
14. Android descuenta, abona y anota los dos movimientos bajo el mismo candado, y responde {movimientoId, nuevoSaldo, destinoNombre, fecha}.
15. El bundle envía TRANSFER_SUCCESS de inmediato y solo después pinta la pantalla de resultado con el nuevo saldo. TransferenciaActivity lo registra en el log: la pantalla la cierra el propio bundle con el botón atrás.
16. Al volver, HomeActivity.onResume reenvía LOAD_HOME porque el saldo cambió y el bundle de Home sigue montado con los datos viejos. No se recarga la pantalla, solo se le manda el dato nuevo.

El aviso va antes de pintar el resultado a propósito. Si el usuario se devuelve de una, Home ya tiene que estar enterada.

4.6 Recorrido 3: la sesión que vence

Hay dos maneras de enterarse de que la sesión venció, y las dos terminan en el mismo sitio.

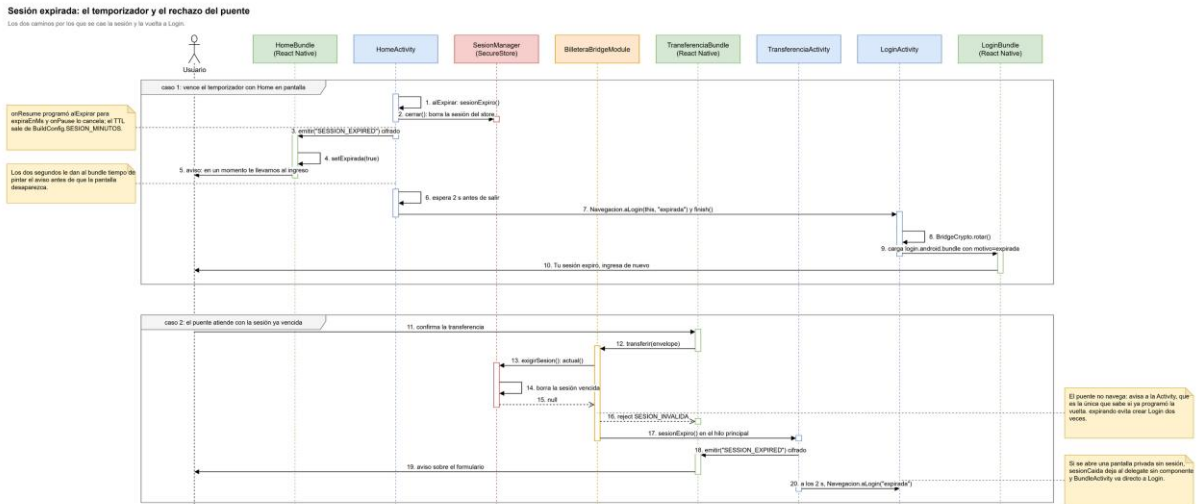


Figura 8. Diagrama de secuencia de la expiración. Arriba, el plazo se cumple con la pantalla al frente. Abajo, el usuario pide algo justo después de que venció.

- **Con la pantalla en primer plano.** En onResume, cada Activity privada programa un Handler para el instante exacto del vencimiento, que es `expiraEnMs` menos el reloj. Al dispararse, llama a `sesionExpira()`.
- **Al intentar una operación.** Cualquier método del puente que exija sesión (`buscarDestino`, `transferir`, `movimientos`) llama a `exigirSesion()`. Si no hay sesión vigente rechaza con `SESSION_INVALIDA` y le avisa a la Activity.

`sesionExpira()` está protegido por una bandera para que las dos rutas no lo ejecuten dos veces. Cierra la sesión, emite `SESSION_EXPIRED` al bundle activo y programa la vuelta a Login dos segundos después. Esos dos segundos existen para que el bundle alcance a pintar el aviso antes de que la pantalla desaparezca. Login recibe motivo: "expirada" en sus props y muestra el mensaje correspondiente.

El Handler se cancela en onPause, incluida la vuelta a Login. Si la pantalla se va con la cuenta regresiva andando, el `startActivity` no sale desde segundo plano pero el `finish()` sí, y la tarea quedaría vacía.

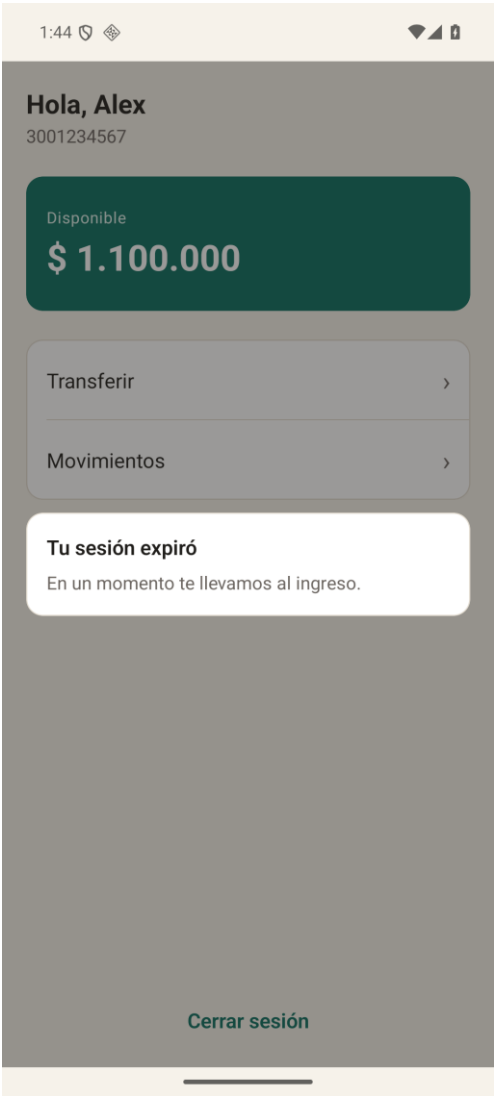


Figura 9. El aviso que pinta el bundle al recibir `SESSION\_EXPIRED`, antes de que Android vuelva a Login.

5. Manejo de sesión

5.1 Qué se guarda

La sesión guardada tiene los cinco campos que pide el enunciado más uno propio.

```
{
  "sessionId": "8c2f7c1e-4d3a-4b6e-9d2f-0a1b2c3d4e5f",
  "userId": "u10001",
  "name": "Alex Suárez",
  "phone": "3001234567",
  "expiresAt": "2026-09-11T22:41:00",
  "expiraEnMs": 1789512060000
}
```

sessionId es un UUID que genera Android con `UUID.randomUUID()`. expiresAt queda en hora local porque es lo que se muestra en pantalla. El vencimiento, en cambio, se compara siempre contra expiraEnMs, que es tiempo epoch en milisegundos, para no depender de la zona horaria ni del formato de fecha. Esa es la única razón por la que existe el sexto campo.

La sesión **no contiene la clave del usuario ni nada derivado de ella**, y hay una prueba unitaria que lo comprueba de forma explícita sobre la respuesta de login.

5.2 Dónde y cómo se guarda

El JSON se serializa con `org.json`, se cifra con la llave AES-256 del Android Keystore y se guarda en SecureStore bajo la clave `sesion`, dentro de un `SharedPreferences` privado llamado `billettera`. El valor escrito tiene esta forma:

```
base64(iv) + "." + base64(ciphertext || tag)
```

La llave no sale nunca del Keystore. SecureStore le pasa bytes al proveedor criptográfico y recibe bytes de vuelta.

5.3 Cuándo se valida

La sesión se revisa en cuatro sitios, y los cuatro hacen falta.

- En `SplashActivity`, para decidir la pantalla de entrada.
- En `onCreate` de toda `Activity` cuyo `RnBundle.privado` sea `true`. Sin sesión, la pantalla ni siquiera monta el bundle: redirige a Login.
- En `onResume` de esas mismas `Activities`, y ahí además se programa el `Handler` del vencimiento.
- En cada método del puente que toque datos del usuario: `buscarDestino`, `transferir` y `movimientos`.

`SesionManager.actual()` borra la sesión si está vencida o si el JSON no se puede leer, así que leer y limpiar son la misma operación y no queda basura en el almacén.

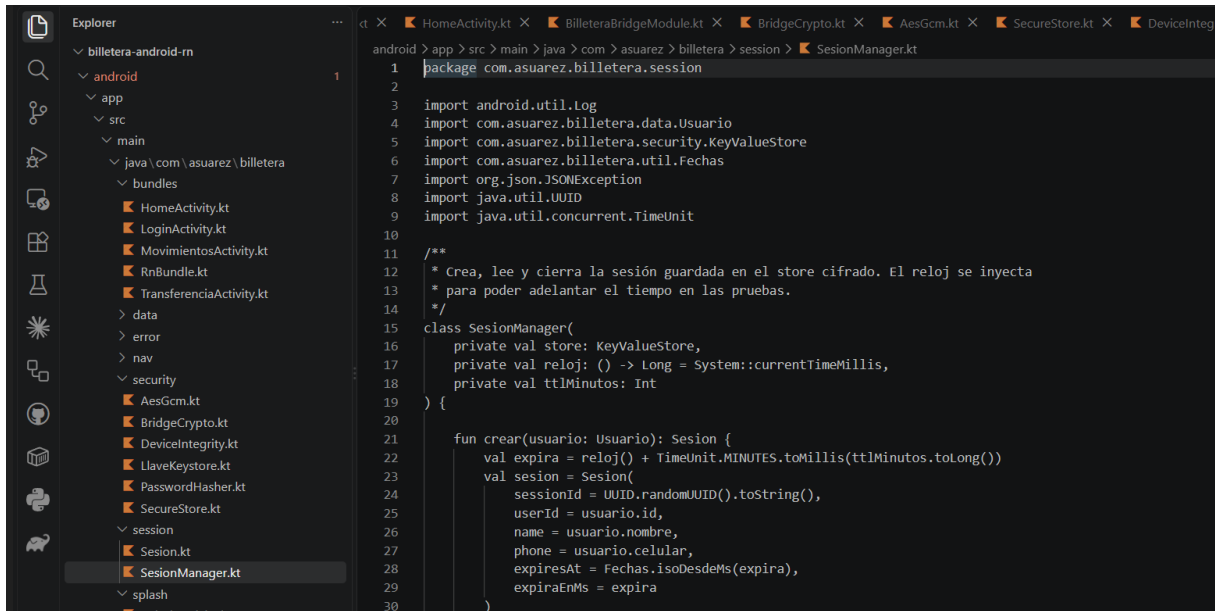


Figura 10. Figura 10. `SesionManager`. El reloj se inyecta por constructor, lo que permite adelantar el tiempo en las pruebas sin tocar el reloj del sistema.

5.4 Cuándo expira

El tiempo de vida sale de BuildConfig.SESION_MINUTOS: **2 minutos en depuración y 10 en release**. Los 2 minutos de depuración son deliberados, para poder demostrar la expiración sin esperar.

El plazo se programa con precisión, no con un sondeo periódico: en onResume se calcula expiraEnMs menos el reloj actual y se agenda un Handler para ese instante exacto.

5.5 Qué pasa al cerrar sesión

El logout es el camino más corto de los dos. HomeActivity recibe el evento LOGOUT y hace cuatro cosas, en este orden:

17. session.cerrar() borra la clave session del almacén cifrado.
18. Navegacion.aLogin(this, "logout") abre Login con NEW_TASK | CLEAR_TASK, lo que destruye Home y con ella su host de React Native.
19. LoginActivity.onCreate detecta el extra motivo y llama BridgeCrypto.rotar() antes de armar las props, de modo que la llave del canal anterior deja de servir.
20. finish() sobre Home.

Pendiente honesto, anotado como TODO en SesionManager: hoy el tiempo de vida es fijo y no se renueva con la actividad del usuario. Es lo que haría un producto real, y no está hecho.

6. Cifrado

Hay dos cifrados distintos en la aplicación y conviene no confundirlos: el del almacenamiento y el del canal hacia JavaScript. Usan llaves diferentes, con ciclos de vida diferentes, y resuelven problemas diferentes.

6.1 El cifrado del almacenamiento

Una llave AES-256 generada dentro del Android Keystore bajo el alias `billetera.store.v1`, con `PURPOSE_ENCRYPT` | `PURPOSE_DECRYPT`, modo GCM y sin relleno. Ahí viven la sesión y el estado del banco simulado.

Si el alias queda inservible, por un keystore corrupto o una llave que el sistema invalidó, se descarta y se regenera. Lo que se pierde es la sesión guardada, no el acceso a la aplicación.

Un detalle de implementación vale la pena señalarlo, porque es el tipo de cosa que uno corrige mal la primera vez: con llaves del Keystore, Android exige generar el vector de inicialización él mismo y rechaza el que uno le pase. Por eso `AesGcm.cifrar` llama `cipher.init(ENCRYPT_MODE, llave)` sin parámetros y lee `cipher.iv` después.

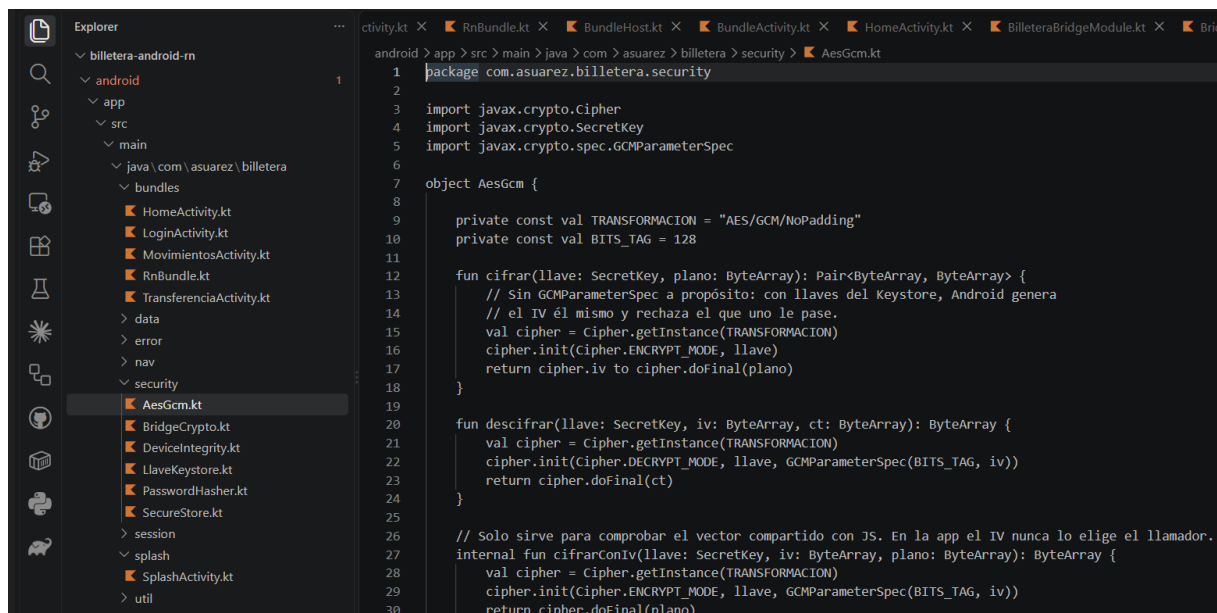


Figura 11. Figura 11. `AesGcm`. El método `cifrarConIv` es `internal` y solo lo usan las pruebas del vector compartido; en la aplicación el vector de inicialización nunca lo elige el llamador.

6.2 El sobre cifrado del canal

El canal hacia JavaScript usa otra llave: 32 bytes de `SecureRandom` generados en `BilleteraApp.onCreate`, que viven solo en memoria. Esa llave se le entrega al bundle en las props iniciales, se rota al volver a Login por logout o por expiración, y no se persiste ni se registra en ningún log.

El formato del sobre es idéntico en los dos lados.

- AES-256-GCM, con vector de inicialización de 12 bytes, aleatorio y distinto por mensaje.
- Tag de autenticación de 128 bits, pegado al final del ciphertext.
- Sin datos autenticados adicionales (AAD).
- Texto plano: el objeto serializado como JSON en UTF-8.

```
{
  "iv":  "<base64 de 12 bytes>",
  "data": "<base64 de ciphertext || tag>"
}
```

Android lo hace con `javax.crypto` y `java.util.Base64`. JavaScript lo hace con `gcm` de `@noble/ciphers` y `base64` de `@scure/base`.

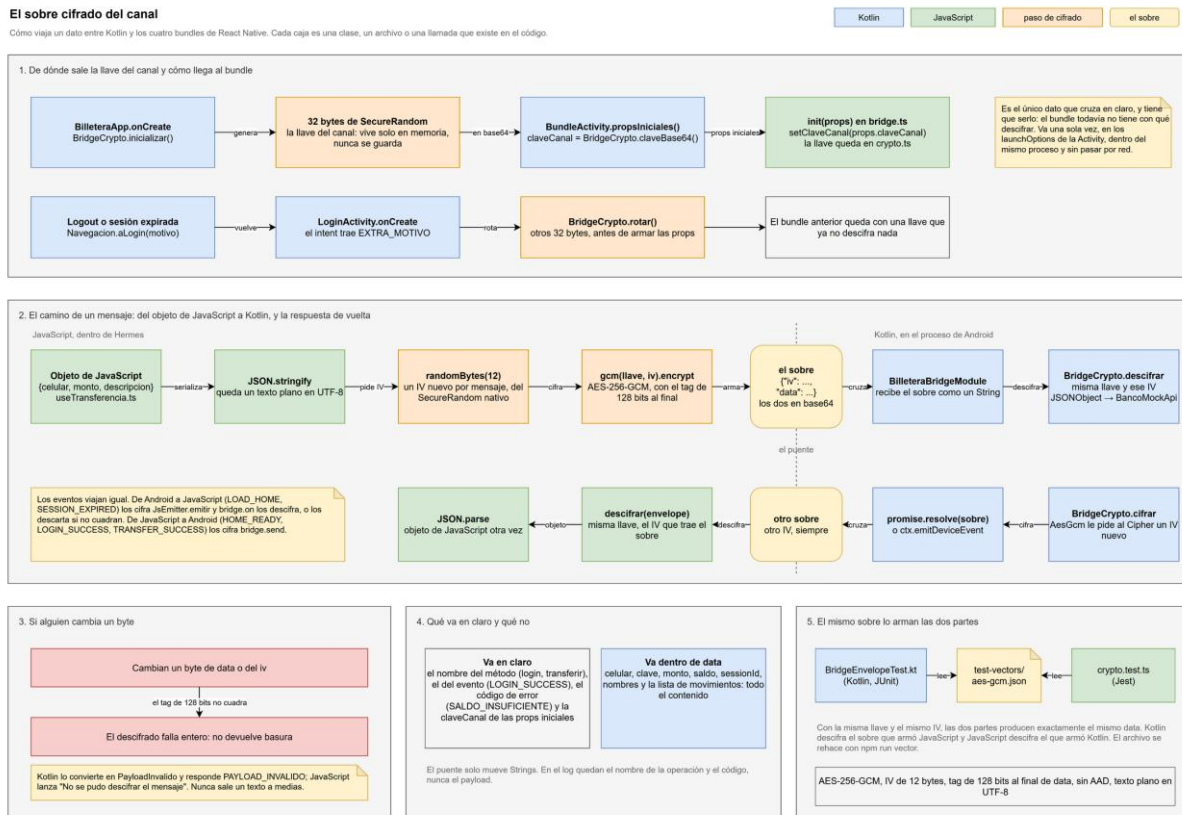


Figura 12. El sobre cifrado del canal: de dónde sale la llave, cómo llega al bundle, el camino de ida y vuelta de un mensaje, y qué queda en claro.

6.3 Interoperabilidad comprobada con un vector compartido

Que las dos implementaciones hablen el mismo formato no queda a la confianza. El archivo `test-vectors/aes-gcm.json` fija una llave, un vector de inicialización y un texto plano, que es justamente un JSON de sesión, y guarda el sobre esperado.

Prueba	Lado	Qué comprueba
BridgeEnvelopeTest	Kotlin, JUnit	Descifra el sobre que produjo JavaScript y comprueba que, al volver a cifrar con el mismo vector de inicialización, sale exactamente el mismo data.
crypto.test.ts	JavaScript, Jest	Lo mismo desde el otro lado, contra el mismo archivo.

Si alguien cambia el modo, el tamaño del tag o el orden de los bytes, esas dos pruebas se caen a la vez. El archivo se regenera con `npm run vector`, y la aplicación no lo lee nunca: esa llave solo existe para las pruebas.

Un tropiezo quedó documentado en el código: Hermes 0.81 no trae `TextDecoder`, así que la función `bytesToUtf8` de la librería no sirve. El decodificador UTF-8 de `crypto.ts` son veinte líneas escritas a mano por ese motivo, y hay un caso de prueba que borra `globalThis.TextDecoder` para asegurarse de que el camino funciona sin él.

6.4 Qué se cifra y qué no

Va dentro del sobre	Va en claro
Credenciales de login y de registro	Los nombres de evento: <code>LOGIN_SUCCESS</code> , <code>LOAD_HOME</code> , y los demás
Datos de sesión y del usuario	Los códigos de error: <code>SALDO_INSUFICIENTE</code> , <code>SESSION_INVALIDA</code> , y los demás
El saldo disponible	La respuesta de <code>randomBytes</code> , que son bytes aleatorios sin significado
Las órdenes de transferencia y su resultado	<code>claveCanal</code> en las props iniciales, que es la excepción obligada
La lista de movimientos y los identificadores	Los otros campos de las props: <code>bundle</code> , <code>esDebug</code> , <code>version</code> y <code>motivo</code>

No hay ninguna llave escrita en el código. La del almacenamiento la genera el Keystore en el primer arranque, la del canal sale de `SecureRandom` en cada arranque del proceso, y la del vector de pruebas vive en un archivo de test que la aplicación no lee. Lo único constante en el código son los hashes PBKDF2 de los usuarios semilla, que son hashes con sal, no claves.

6.5 Modelo de amenazas del canal

Esta parte va completa, porque un cifrado mal explicado es peor que no tenerlo.

Qué sí protege

- Los payloads no viajan en claro por el puente. Eso cierra la ventana de quien inspecciona el tráfico del bridge con las herramientas normales de desarrollo: un Metro conectado, un menú de desarrollador, un registrador de mensajes del puente, un volcado de los argumentos de los métodos nativos.
- Cierra también el registro accidental. Si algo se escribe por error en el log, lo que sale es un sobre, no un saldo ni una clave.
- La llave es efímera por proceso, así que un volcado de memoria viejo no sirve para leer una sesión nueva.
- La llave se rota al volver a Login, de modo que un bundle que quedó con la llave anterior no puede leer nada de la sesión siguiente.
- No hay secretos quemados en el APK: descompilarlo no entrega ninguna llave.

Qué no protege

Android y JavaScript son el mismo proceso. La llave del canal está en la memoria de ese proceso y se le entrega al bundle en sus props. Con root y un instrumentador dinámico se lee la llave, o directamente el texto plano antes de cifrarlo y después de descifrarlo, sin necesidad de romper nada criptográfico.

Contra un atacante con control del dispositivo, este cifrado no es una barrera: es higiene. Tampoco protege de un bundle malicioso, porque un bundle que se ejecuta dentro de la aplicación ya tiene la llave por diseño. La integridad del bundle la da que esté dentro del APK firmado, no el cifrado.

Cuál sería el siguiente paso

Dejar de entregar la llave y acordarla. Un intercambio ECDH efímero entre el módulo nativo y el bundle, con la llave de sesión derivada de ese intercambio, para que la llave completa no exista nunca en las props ni en un solo lugar.

Y para la parte que este esquema no puede resolver, atestación del dispositivo y de la aplicación con Play Integrity, que responde la pregunta de verdad, es decir si el proceso está corriendo donde debería, en vez de adivinarla con heurísticas locales como las de hoy.

7. Seguridad implementada, punto por punto

El enunciado trae una lista de once requisitos de seguridad. La tabla los recorre en el mismo orden.

Requisito del enunciado	Cómo quedó implementado	Dónde
Android Keystore	LlaveKeystore genera y guarda una AES-256 bajo el alias <code>billetera.store.v1</code> , con modo GCM y sin relleno. La llave no se exporta. Si el alias queda inservible, se descarta y se regenera.	<code>security/LlaveKeystore.kt</code>
EncryptedSharedPreferences o equivalente	Aquí está el equivalente. SecureStore cifra cada valor con la llave del Keystore antes de escribirlo en un SharedPreferences privado. Se escribió propio porque EncryptedSharedPreferences está obsoleto desde security-crypto 1.1 y no tiene reemplazo directo.	<code>security/SecureStore.kt</code>
Cifrado de payloads sensibles	Sobre AES-256-GCM para todo lo que lleve datos de negocio, en los dos sentidos y en los tres mecanismos del puente.	<code>security/BridgeCrypto.kt</code> , <code>src/shared/crypto.ts</code>
Validación de sesión	Splash, onCreate y onResume de las Activities privadas, y cada método del puente que toque datos del usuario.	<code>BundleActivity.kt</code> , <code>BilleteraBridgeModule.kt</code>
Expiración de sesión	Tiempo de vida de 2 minutos en depuración y 10 en release, comparado contra tiempo epoch, con Handler programado al instante exacto y aviso al bundle antes de redirigir.	<code>session/SesionManager.kt</code>
Logout seguro	Borra la sesión del almacén, rota la llave del canal, destruye Home y su host con CLEAR_TASK y vuelve a Login.	<code>HomeActivity.kt</code> , <code>LoginActivity.kt</code>
No secretos quemados	Ninguna llave en el código. Lo único constante son los hashes PBKDF2 de los usuarios semilla, que son hashes	<code>data/DatosSemilla.kt</code>

	con sal.	
No contraseñas en texto plano	PBKDF2WithHmacSHA256 con sal de 16 bytes, 256 bits de salida y 210.000 iteraciones, en el formato <code>pbkdf2\${iter}\${sal}\${hash}</code> , comparado con <code>MessageDigest.isEqual</code> . La clave no se guarda, no se devuelve, no entra al JSON de sesión y no aparece en ningún log.	<code>security/PasswordHasher.kt</code>
Protección básica contra root y emulador	<code>DeviceIntegrity</code> revisa nueve rutas de su <code>Build.TAGS</code> con <code>test-keys</code> , tres gestores de root instalados, <code>which su</code> , y siete señales de emulador. Bloquea solo si hay root, no es emulador y la compilación es de release.	<code>security/DeviceIntegrity.kt</code>
Ofuscación con ProGuard/R8	<code>minifyEnabled true</code> y <code>shrinkResources true</code> en release, con <code>mapping.txt</code> generado. Las reglas propias son mínimas porque <code>react-android</code> ya trae sus reglas consumer para el puente.	<code>android/app/build.gradle</code> , <code>proguard-rules.pro</code>
Manejo controlado de errores	Ningún <code>Throwable</code> llega a la interfaz ni a JavaScript. Los métodos del puente envuelven todo en <code>runCatching</code> ; los errores esperados rechazan con su código y los inesperados con <code>ERROR_INTERNO</code> y un registro que solo guarda el nombre de la clase de la excepción.	<code>BilleteraBridgeModule.kt</code> , <code>error/ErrorActivity.kt</code>

7.1 Notas sobre tres de esos puntos

Las iteraciones de PBKDF2. OWASP sugiere 600.000 para un servidor. Aquí el hash corre en el teléfono y con 210.000 el login se queda por debajo de medio segundo en los equipos donde se probó. La constante está en un solo sitio y el formato guardado lleva el número de iteraciones, así que subirlo más adelante no invalida los hashes viejos.

La política de root. Las imágenes `google_apis` del emulador son compilaciones `userdebug`, o sea que la aplicación las detecta como equipo modificado. Si bloqueara ahí, nadie podría evaluar la

prueba. Por eso la política es: se bloquea solo si hay root, no es emulador y la compilación es de release. En el emulador la aplicación muestra las señales y deja continuar; en un teléfono real con señales de root, el release se cierra.

La detección de root es heurística y se sabe. Binarios su en las rutas conocidas, test-keys en Build.TAGS, tres paquetes de gestores conocidos y un par de propiedades del sistema. Un Magisk bien escondido pasa sin que la aplicación se entere, y eso no se arregla agregando más rutas a la lista. La respuesta real es Play Integrity, y así está anotado en el código.

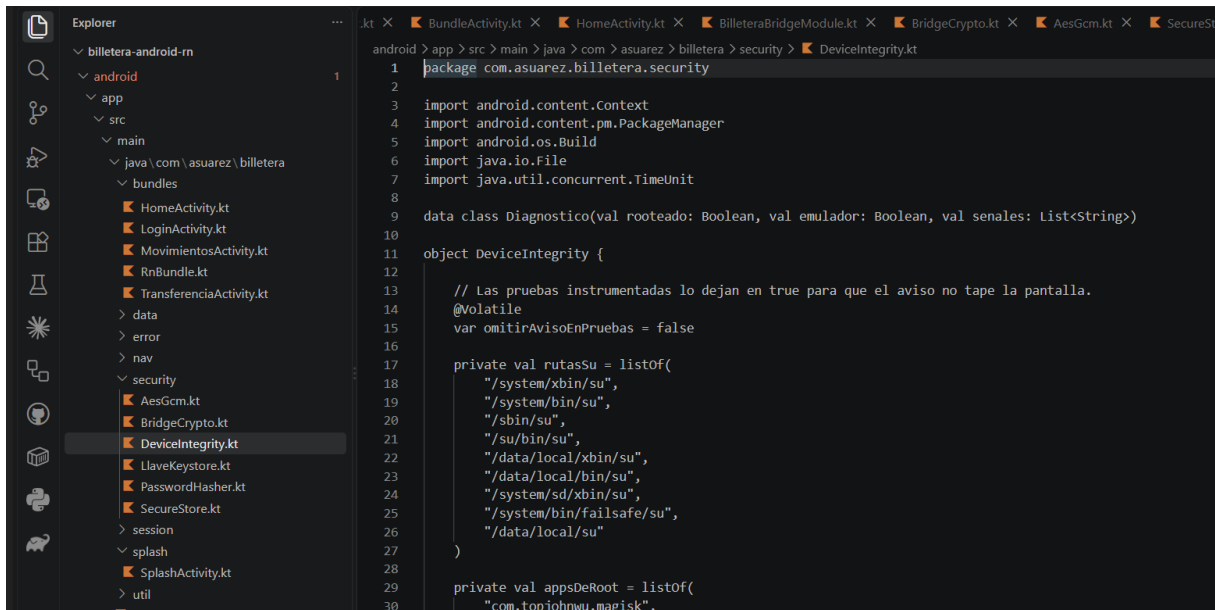


Figura 13. Figura 13. 'DeviceIntegrity'. El bloque '<queries>' del manifest es indispensable: sin él, 'getPackageInfo' no ve ningún paquete desde Android 11 y la detección por gestores instalados no sirve de nada.

7.2 Dos medidas adicionales que no estaban en la lista

'FLAG_SECURE' en release. Las cuatro Activities de bundle agregan esa bandera cuando la compilación no es de depuración, lo que impide capturas de pantalla y grabación. En depuración no se aplica, precisamente para poder tomar las capturas de la evidencia.

RespalDOS apagados. El manifest declara allowBackup="false" y un archivo de reglas de extracción que deja las SharedPreferences por fuera de la transferencia entre dispositivos, que desde Android 12 se las lleva igual aunque el respaldo en la nube esté apagado.

8. El backend simulado y sus reglas de negocio

8.1 Cómo está construido

BancoMockApi es un backend que corre en el mismo proceso de la aplicación. Está escrito como se escribiría un cliente HTTP: métodos que devuelven `Resultado.Ok` o `Resultado.Error(codigo, mensaje)`, sin excepciones para el flujo normal.

La latencia simulada de 200 a 400 ms en autenticar y transferir obliga a las pantallas a manejar estados de carga de verdad. Corre siempre en el hilo de módulos nativos, nunca en el de interfaz, y se puede apagar por constructor para las pruebas.

El estado vive en MockDb: usuarios y movimientos serializados con `org.json` y guardados en el mismo almacén cifrado, bajo la clave `mockdb`. La carga es perezosa; si no hay nada guardado o el JSON no se puede leer, siembra con `DatosSemilla`. Todas las operaciones son `@Synchronized` y guardan después de cada cambio.

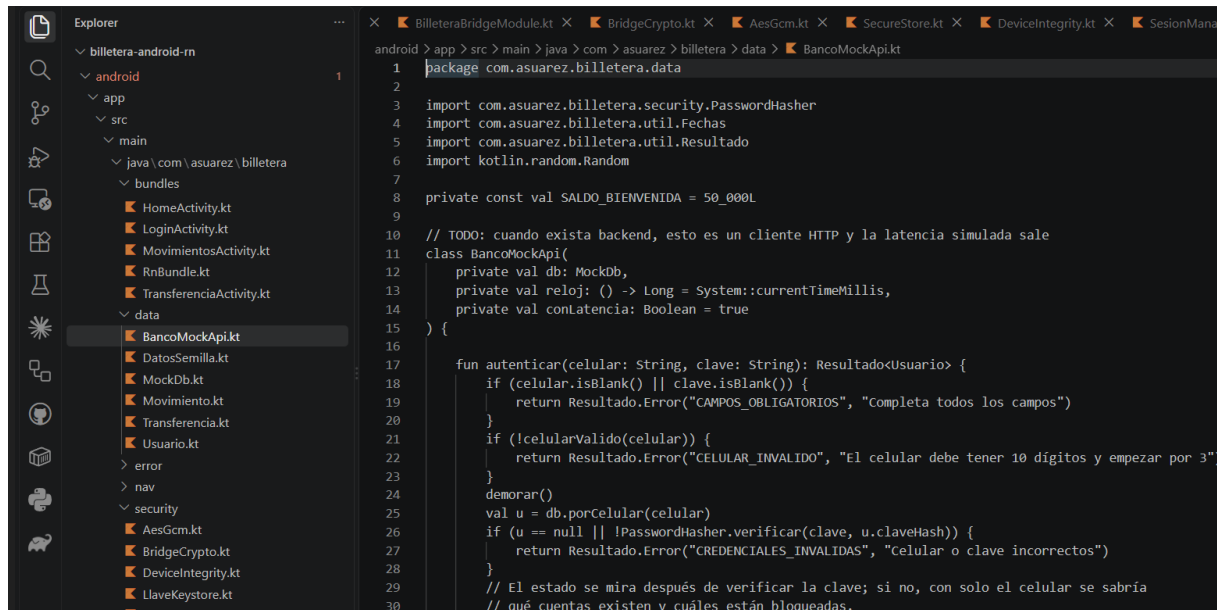


Figura 14. Figura 14. 'BancoMockApi'. El TODO de la cabecera dice lo que corresponde: cuando exista backend, esto es un cliente HTTP y la latencia simulada sale.

8.2 Las reglas

Login

Campos obligatorios, celular de 10 dígitos que empiece por 3, y clave verificada contra el hash. El estado del usuario se revisa **después** de verificar la clave, no antes. Si se revisara antes, con solo el celular se sabría qué cuentas existen y cuáles están bloqueadas.

Un usuario INACTIVO devuelve USUARIO_INACTIVO, que es una decisión de experiencia de usuario: le dice al titular legítimo qué le está pasando. En producción se unificaría con "credenciales inválidas" y el backend limitaría los intentos.

Registro

Celular libre, clave de al menos 6 caracteres. Crea el usuario ACTIVO con identificador correlativo, u10004 y siguientes, y **saldo inicial de 50.000 con un movimiento CREDITO "Abono de bienvenida"**. Es un comportamiento del mock, no una regla de negocio real, pero deja que una cuenta recién creada pueda hacer algo sin tener que sembrarla a mano.

Transferencia

Sesión válida, celular destino con formato correcto, monto entero mayor que cero, destino existente y ACTIVO, y no a sí mismo.

La verificación del saldo y el descuento ocurren dentro del mismo método sincronizado, MockDb.aplicarTransferencia, no en el llamador. Si se comprobara afuera, la demora simulada dejaría el dato viejo y dos transferencias al tiempo podrían dejar la cuenta en rojo.

Una transferencia exitosa deja dos movimientos con la misma fecha: uno DEBITO en el origen ("Transferencia a Laura P.", con la descripción del usuario anexada si la escribió) y uno CREDITO en el destino ("Transferencia de Alex S."). Los dos se escriben con un solo guardado, para que no quede plata abonada sin su débito.

Si el saldo no alcanza, el mock anota un movimiento DEBITO en estado RECHAZADA con la descripción "Transferencia rechazada: saldo insuficiente", no descuenta nada y responde SALDO_INSUFICIENTE. El intento queda en el extracto, que es lo que uno espera ver al revisar la cuenta.



Figura 15. Rechazo por saldo insuficiente. La hoja de confirmación se cierra y el monto queda editable.

Movimientos

Solo los del usuario autenticado, del más reciente al más antiguo, con fecha, tipo, valor, descripción y estado. La agrupación por día la hace el bundle en memoria.

8.3 Datos semilla

id	Nombre	Celular	Estado	Saldo
u10001	Alex Suárez	3001234567	ACTIVO	1.250.000
u10002	Laura Pérez	3109876543	ACTIVO	480.000
u10003	Carlos Gómez	3157654321	INACTIVO	90.000

Los tres comparten la misma clave de prueba, que está en el README. En el dispositivo solo queda el hash PBKDF2. Carlos existe para poder demostrar el usuario inactivo, tanto al ingresar como al recibir una transferencia.

Alex arranca con dos movimientos, un abono de nómina de 1.500.000 y un pago de servicios de 250.000, y Laura con uno, todos con fechas fijas de la semana anterior. Los identificadores de movimiento se generan con `Locale.ROOT` fijo, porque con el locale del sistema en árabe el formato `%05d` sale con otros dígitos y dos movimientos podrían terminar con el mismo id.

8.4 Una nota sobre enumeración de cuentas

`buscarDestino` responde si un celular tiene cuenta y devuelve el nombre corto del titular. Eso es exactamente lo que necesita la pantalla de transferencia para que el usuario confirme a quién le está mandando plata, y es lo que hacen las billeteras reales. También significa que un usuario autenticado puede recorrer el espacio de números y averiguar qué celulares tienen cuenta y cómo se llama más o menos su dueño.

No está resuelto, y es mejor decirlo que fingir que no está. En un producto real esto se mitiga del lado del servidor, no del cliente: límite de consultas por sesión y por ventana de tiempo, bloqueo progresivo tras varios fallos seguidos, y devolver el nombre en la forma más corta posible. Mostrar solo iniciales ayuda poco, porque lo que se filtra es la existencia de la cuenta, no el nombre.

La decisión de producto de fondo es si se confirma el destinatario antes o después de escribir el monto. Aquí se confirma antes, porque equivocarse de número es el error más común y el más caro.

9. Estrategia de pruebas

Tres niveles, escogidos por lo que cada uno puede comprobar de verdad. Las salidas completas están en docs/evidencia.md.

Nivel	Comando	Resultado	Qué cubre
Kotlin en la JVM	<code>gradlew testDebugUnitTest</code>	40 pruebas, 0 fallos, 0 errores, 1 omitida	AES-GCM y el vector compartido, el formato del sobre, PBKDF2, el ciclo de la sesión con reloj falso y las reglas del banco simulado, con 17 casos.
Kotlin instrumentado en emulador	<code>gradlew connectedDebugAndroidTest</code>	13 pruebas en 8 clases, todas en verde	Round trip real contra el Keystore, persistencia de la sesión, redirección del splash, carga efectiva de los bundles de Login y Home, redirección de Home sin sesión, apertura de Transferencia y Movimientos, y logout.
Jest	<code>npm test</code>	8 suites, 43 pruebas, todas en verde	Cifrado (round trip, vector compartido y el caso sin TextDecoder), validaciones, formatos, agrupación por día, la máquina de pasos de la transferencia y tres suites de componentes.

La prueba omitida es deliberada: PasswordHasherTest incluye el generador de los hashes semilla, que solo corre con `-Dsemilla=true` para no rehacer PBKDF2 en cada compilación.

A esos tres niveles se suman `npm run typecheck` y `npm run lint`, los dos en cero errores, y la compilación y prueba de humo del APK de release firmado y ofuscado sobre el emulador.

9.1 Cobertura de las once pruebas mínimas

El enunciado lista once pruebas mínimas. Las once están cubiertas por al menos una prueba automática, y docs/evidencia.md trae la tabla de correspondencia una por una, con el nombre exacto de cada caso y la captura que le corresponde.

El cifrado es el único punto que no se ve en una captura, porque el canal va cifrado de punta a punta y en pantalla se ve el resultado, no el sobre. La prueba de que los dos lados hablan el mismo formato es el vector compartido descrito en la sección 6.3. Del lado del almacenamiento,

SecureStoreKeystoreTest guarda un valor, lo lee crudo de las SharedPreferences y comprueba que ahí no está el texto plano.

9.2 Lo que no se pudo probar

- **Capturas del APK de release.** Con FLAG_SECURE activo, el compositor de pantalla devuelve la ventana en negro. El flujo de release se verificó igual, pantalla por pantalla, leyendo el árbol de interfaz con uiautomator y revisando logcat, sin excepciones ni cierres.
- **Transferencia de punta a punta automatizada en el dispositivo.** La regla de negocio está cubierta en la JVM y la máquina de pasos en Jest; en el emulador se hizo a mano. Espresso contra vistas de React Native da pruebas que fallan por tiempos más que por código.
- **Bloqueo por root en un equipo físico.** No hubo un teléfono rooteado disponible, así que la rama que cierra la aplicación nunca llegó a ejecutarse. La otra mitad de la política, el aviso con señales sobre el emulador, sí quedó probada.
- **Un solo dispositivo.** Todo corrió sobre API 36, x86_64. No se probó en API 26, que es el minSdk declarado, ni en arm64.

10. Limitaciones conocidas y próximos pasos

10.1 Limitaciones

Limitación	Alcance real
El backend es un mock en proceso	BancoMockApi resuelve todo en el dispositivo y MockDb guarda el estado en las mismas SharedPreferences cifradas. No hay red, y el APK de release ni siquiera pide el permiso de INTERNET.
La llave del canal viaja en las props	Cubre lo que pide el enunciado, es decir que los payloads no queden legibles en un log, en un volcado o en las herramientas que leen el tráfico del puente, pero no cubre a alguien que ya controla el proceso: si lo controla, también tiene la llave.
El tiempo de vida de la sesión es fijo	No se renueva con la actividad del usuario. Está anotado como TODO en SesionManager.
La expiración se mide contra el reloj del dispositivo	Cambiar la hora del teléfono alarga o acorta la sesión. Con backend real, el vencimiento lo diría el servidor.
Movimientos no pagina	Trae la lista completa del usuario y agrupa por día en memoria. Con el mock da igual; con historial real, no.
La detección de root es heurística	Un Magisk bien escondido pasa sin que la aplicación se entere, y eso no se arregla agregando más rutas a la lista.
buscarDestino permite enumerar cuentas	Descrito en la sección 8.4. La mitigación es del lado del servidor, no del cliente.

10.2 Próximos pasos, en orden

21. **Backend real.** BancoMockApi ya tiene la forma de un cliente: cambiar MockDb por HTTP con TLS y fijación de certificado es un reemplazo, no una reescritura. Con eso desaparecen la latencia simulada y la base local.
22. **Acordar la llave del canal con ECDH** en vez de entregarla en las props, y atestación con Play Integrity en el arranque en lugar de las heurísticas locales de root.
23. **Sesión que se renueva con la actividad**, con refresh token del lado del servidor, en vez del tiempo de vida fijo de hoy.
24. **Biometría** para volver a entrar sin escribir la clave, apoyada en una llave del Keystore con setUserAuthenticationRequired.
25. **Paginación de movimientos**, que hoy trae la lista completa.
26. **Migrar a la nueva arquitectura** cuando el patrón de un host por bundle tenga una respuesta clara en bridgeless, y quitar las supresiones de obsolescencia.

10.3 Dos desviaciones respecto de lo planeado

Dos cosas quedaron distintas de como estaban diseñadas, y las dos por el mismo motivo: al escribirlas contra el código real de React Native 0.81.6 no funcionaban. Quedan aquí porque explican decisiones del código que de otro modo parecen arbitrarias.

El redirect por sesión inválida va después de `super.onCreate`, no antes. El plan era comprobar la sesión, redirigir a Login y salir sin llamar nunca a `super.onCreate`, para que el bundle no se cargara. Leyendo `ReactActivityDelegate` se ve por qué eso no sirve: `onPause` y `onDestroy` hacen `Objects.requireNonNull(mReactDelegate)`, y ese campo solo se inicializa dentro del `onCreate` del delegate. Saltarse el `super` deja el delegate en null y la Activity falla al cerrarse.

La salida está en la misma clase: el delegate solo monta el surface if (`mainComponentName != null`). Con el nombre del componente en null, arma toda su maquinaria pero no carga el bundle. Así que `BundleDelegate.getMainComponentName()` devuelve null cuando la sesión cayó, `super.onCreate` corre completo sin tocar JavaScript, y el redirect ocurre inmediatamente después. El resultado visible es el mismo, porque con sesión inválida no se ejecuta una línea del bundle, y el ciclo de vida queda sano.

Los insets se aplican al contenido de la Activity, no con `fitsSystemWindows` en el `ReactRootView`. Desde `targetSdk 35` la ventana va siempre de borde a borde, así que hay que separar el contenido de la barra de estado y de la de navegación. El plan era poner `fitsSystemWindows = true` en el `ReactRootView`. No funciona: en arquitectura clásica el `UIManager` posiciona los hijos del root view por su cuenta, con coordenadas absolutas, y se salta el padding del contenedor.

Lo que sí respeta es el padding del `android.R.id.content` de la Activity, así que `BundleActivity.separarDeLasBarras()` instala ahí un `OnApplyWindowInsetsListener`. Esto tiene una consecuencia del lado de JavaScript: como la vista de React arranca ya debajo de la barra de estado y la altura del teclado llega en coordenadas de pantalla, el `KeyboardAvoidingView` de `Pantalla.tsx` compensa con `keyboardVerticalOffset={StatusBar.currentHeight}`. Sin eso, el teclado tapa el final del formulario.

Anexo. Índice de figuras y archivos de referencia

Figura	Archivo en el repositorio
Figura 1. Diagrama de componentes	docs/diagramas/componentes.png
Figura 5. Secuencia de inicio de sesión	docs/diagramas/secuencia-login.png
Figura 7. Secuencia de transferencia	docs/diagramas/secuencia-transferencia.png
Figura 8. Secuencia de sesión expirada	docs/diagramas/secuencia-sesion-expirada.png
Figura 12. Sobre cifrado del canal	docs/diagramas/cifrado-canal.png

Cada diagrama tiene además su fuente editable junto al PNG, en formato HTML o .drawio.

Documento	Contenido
README.md	Cómo compilar y correr, usuarios de prueba, organización del repositorio y resumen de decisiones.
docs/arquitectura.md	Versión en Markdown de este documento, dentro del repositorio.
docs/evidencia.md	Evidencia de pruebas: correspondencia con las once pruebas mínimas, salidas completas de los tres niveles y el recorrido en capturas.
test-vectors/aes-gcm.json	Vector compartido de AES-256-GCM que comprueba la interoperabilidad entre Kotlin y JavaScript.